OS

Unit 4 assignment

Name: Rehana N
SRN:PES2UG24CS678
SECTION:4k

PES Version Control System Lab

1. Introduction Version control systems are essential tools for tracking changes in software development. This project implements a simplified version control system inspired by Git, focusing on core concepts such as content-addressable storage, staging, and commit history. The system stores complete snapshots of the project rather than differences, ensuring data integrity and efficient storage through hashing and deduplication.

2. System Overview

The PES-VCS system consists of the following components: Working Directory: Contains user files Index (Staging Area): Tracks files staged for the next commit Object Store: Stores all data (blobs, trees, commits) using hash-based addressing References: Maintains commit history and branch pointers

3. Phase 1: Object Storage

Concepts Covered:

Content-addressable storage

SHA-256 hashing

Data deduplication

Atomic file operations

Implementation:

The object storage system stores file contents as objects identified by their SHA-256

hash. Each object is stored in a directory structure based on the first two characters of

the hash to improve filesystem efficiency.

The object_write() function:

Adds a header (type size\0) to data

Computes SHA-256 hash

Stores the object using atomic write operations

The object_read() function:

Reads stored objects

Verifies integrity using hash comparison

Extracts original data

Screenshot 1A: Output of `./test_objects` showing all tests passing.

```
rehan@REHANA:~/PES2UG24CS678-pes-vcs$ make test_objects
./test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
rehan@REHANA:~/PES2UG24CS678-pes-vcs$
```

Screenshot 1B: `find .pes/objects -type f` showing the sharded directory structure.

```
rehan@REHANA:~/PES2UG24CS678-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
rehan@REHANA:~/PES2UG24CS678-pes-vcs$
```

Screenshot 2A: Output of `./test_tree` showing all tests passing.

```
rehan@REHANA:~/PES2UG24CS678-pes-vcs$ make test_tree
./test_tree
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
rehan@REHANA:~/PES2UG24CS678-pes-vcs$
```

Screenshot 2B: Pick a tree object from `find .pes/objects -type f` and run `xxd .pes/objects/XX/YYY... | head -20` to show the raw binary format.

```
rehan@REHANA:~/PES2UG24CS678-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
rehan@REHANA:~/PES2UG24CS678-pes-vcs$ xxd .pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476 | he
ad -20
00000000: 626c 6f62 2031 3800 4475 706c 6963 6174  blob 18.Duplicat
00000010: 6520 636f 6e74 656e 740a                 e content.
rehan@REHANA:~/PES2UG24CS678-pes-vcs$
```

Screenshot 3A: Run `./pes init`, `./pes add file1.txt file2.txt`, `./pes status` — show the output.

```
rehan@REHANA:~/PES2UG24CS678-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  object.c
  untracked:  Makefile
  untracked:  tree.c
  untracked:  commit.c
  untracked:  index.c
  untracked:  test_tree.c
  untracked:  test_objects.c
  untracked:  pes.h
  untracked:  tree.h
  untracked:  commit.h
  untracked:  INDEX.C
  untracked:  test_sequence.sh
  untracked:  pes.c
  untracked:  README.md
  untracked:  .gitignore
  untracked:  index.h
```

Screenshot 3B: `cat .pes/index` showing the text-format index with your entries.

```
rehan@REHANA:~/PES2UG24CS678-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776841071 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776841071 6 file2.txt
rehan@REHANA:~/PES2UG24CS678-pes-vcs$
```

Screenshot 4A: Output of `./pes log` showing three commits with hashes, authors, timestamps, and messages.

```
rehan@REHANA:~/PES2UG24CS678-pes-vcs$ ./pes init

echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"
Initialized empty PES repository in .pes/
Committed: 5e76f222656e... Initial commit
Committed: 23cba7a0c5a9... Add world
Committed: bd28157e89de... Add farewell
rehan@REHANA:~/PES2UG24CS678-pes-vcs$ ./pes log
commit bd28157e89de0bb0a7d5c9d1044cb7e8f60e812f4225945ee6cfa9e41583145e
Author: Rehana N <PESXUG24CS678>
Date:   1776842145

    Add farewell
d fQ

commit 23cba7a0c5a956c74a9f554aa9aa56144c38c6547d437bfebbc422a80a3fbe0a
Author: Rehana N <PESXUG24CS678>
Date:   1776842145

    Add world
ll
d fQ

commit 5e76f222656ec0eaf1b95c6ebbf3ccd3115979e4ac88d0249272a29da9241ee8
Author: Rehana N <PESXUG24CS678>
Date:   1776842145

    Initial commit
ial commiQ

rehan@REHANA:~/PES2UG24CS678-pes-vcs$ |
```

Screenshot 4B: `find .pes -type f | sort` showing object store growth after three commits.

```
rehan@REHANA:~/PES2UG24CS678-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/23/cba7a0c5a956c74a9f554aa9aa56144c38c6547d437bfebbc422a80a3fbe0a
.pes/objects/2f/5accb985565d470bd4214e948bc40d4a4327b1796efa60f79c4d7501c509ec
.pes/objects/5e/76f222656ec0eaf1b95c6ebbf3ccd3115979e4ac88d0249272a29da9241ee8
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6f/4c5b8d338583d89d0b4f26d5a17972b93b8974889f938f42e024a91642a539
.pes/objects/bd/28157e89de0bb0a7d5c9d1044cb7e8f60e812f4225945ee6cfa9e41583145e
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/ed/60809e9950ab7d1d3cfb4f3ce502c0ea99337942e01c62889161df0e5f7783
.pes/refs/heads/main
rehan@REHANA:~/PES2UG24CS678-pes-vcs$ |
```

Screenshot 4C: `cat .pes/refs/heads/main` and `cat .pes/HEAD` showing the reference chain.

```
rehan@REHANA:~/PES2UG24CS678-pes-vcs$ cat .pes/refs/heads/main
bd28157e89de0bb0a7d5c9d1044cb7e8f60e812f4225945ee6cfa9e41583145e
rehan@REHANA:~/PES2UG24CS678-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
rehan@REHANA:~/PES2UG24CS678-pes-vcs$
```

```
rehan@REHANA:~/PES2UG24CS678-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 5c3edbb7745d... Initial commit

Log after first commit:
commit 5c3edbb7745d41bd47b1d5e290ead715c2e40b432f7aca40c4ca26c017ada25f
Author: Rehana N <PESXUG24CS678>
Date:   1776842666

    Initial commit
ial commiQ


--- Second Commit ---
Committed: f98c3399a73e... Update file.txt

--- Third Commit ---
Committed: db7ca89b15fc... Add farewell

--- Full History ---
commit db7ca89b15fc4419314fa1ff58ba98e6e61aaec1223ac83dbf8c4c7eb347cf77
Author: Rehana N <PESXUG24CS678>
Date:   1776842667
```

```
    Add farewell
d fQ

commit f98c3399a73e8315b6b36fa474edc6e77c5d169ef1736b02493c9f9b1849ec19
Author: Rehana N <PESXUG24CS678>
Date:   1776842667

    Update file.txt
Q

commit 5c3edbb7745d41bd47b1d5e290ead715c2e40b432f7aca40c4ca26c017ada25f
Author: Rehana N <PESXUG24CS678>
Date:   1776842666

    Initial commit
ial commiQ


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
db7ca89b15fc4419314fa1ff58ba98e6e61aaec1223ac83dbf8c4c7eb347cf77

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/49/0b8046918fb7be050b0af121cb6bab2541fe00d65de0173d62ddaa58ae7e0e
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/5c/3edbb7745d41bd47b1d5e290ead715c2e40b432f7aca40c4ca26c017ada25f
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/bf/0fea3ff145b41b70b6eb091e9ab8ef2a2d60b26006dfc34b83421c68e6cd93
.pes/objects/cb/82a18c3817d6448b14d601807ffc642bd956469ec25672034cb90400d0ec2a
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/db/7ca89b15fc4419314fa1ff58ba98e6e61aaec1223ac83dbf8c4c7eb347cf77
.pes/objects/f9/8c3399a73e8315b6b36fa474edc6e77c5d169ef1736b02493c9f9b1849ec19

=== All integration tests completed ===
rehan@REHANA:~/PES2UG24CS678-pes-vcs$
```

Branching and Checkout

Q5.1: A branch in Git is just a file in `.git/refs/heads/` containing a commit hash. Creating a branch is creating a file. Given this, how would you implement `pes checkout <branch>` — what files need to change in `.pes/`, and what must happen to the working directory? What makes this operation complex?
Ans: A branch in PES is stored as a file in `.pes/refs/heads/<branch>` containing a commit hash. The `HEAD` file points to the current branch.

To implement `pes checkout <branch>`, the following steps are required:

1. **Update HEAD**
   Modify `.pes/HEAD` to point to the new branch:

   ```
   ref: refs/heads/<branch>
   ```

2. **Read target commit**
   Read the commit hash from `.pes/refs/heads/<branch>` and load the corresponding commit object.
3. **Reconstruct working directory**
   - Extract the root tree hash from the commit
   - Recursively read tree objects
   - Recreate directories and files by writing blob contents
   - Remove files that are not present in the target tree
4. **Update index**
   The index should be updated to match the checked-out tree.

Complexity:
This operation is complex because it involves recursive tree traversal, careful file deletion/creation, and ensuring that user changes are not accidentally overwritten.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Ans: A dirty working directory occurs when tracked files have been modified but not committed.

To detect conflicts using only the index and object store:

1. For each entry in the index:
   - Read stored metadata (hash, size, modification time)
2. Compare with working directory:
   - Use `stat()` to check file size and modification time
   - If different, recompute the file's hash
3. Compare hashes:

   ```
   working file hash ≠ index hash → file is modified
   ```

4. Check against target branch:
   - If the same file differs in the target branch's tree

Conflict condition:
If a file is modified in the working directory and also differs in the target branch, checkout must be refused.

Reason:
This prevents overwriting uncommitted user changes, avoiding data loss.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?
Ans: A detached HEAD state occurs when `.pes/HEAD` directly contains a commit hash instead of a branch reference.

Behavior:

- New commits can still be created
- However, no branch points to these commits
- They become **unreachable** from any branch

Risk:

- These commits may be lost during garbage collection

Recovery:

- The user can create a new branch pointing to the commit:

  ```
  pes branch <new-branch>
  ```

- Or manually write the commit hash into `.pes/refs/heads/<branch>`

This makes the commits reachable again.

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.
Ans: Over time, the object store may contain unreachable objects. Garbage collection removes them.

Algorithm:

1. **Initialize reachable set**
   - Start from all branch heads (`.pes/refs/heads/*`)
   - Include HEAD
2. **Traverse object graph**
   - For each commit:
     - mark commit
     - follow parent links
     - follow tree pointer
   - For each tree:
     - mark tree

        - follow entries (subtrees/blobs)
3. **Mark phase**
    - Store all reachable object hashes in a **hash set**
4. **Sweep phase**
    - Iterate through `.pes/objects/`
    - Delete objects not in the reachable set

Data structure:
A **hash set** is used for efficient O(1) lookup of reachable objects.

Estimation:
For 100,000 commits and 50 branches:

- All commits may be reachable → ~100,000
- Including trees and blobs → ~300,000–500,000 objects

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?
Ans: Running garbage collection concurrently with a commit operation is dangerous due to race conditions.

Scenario:

1. A commit operation creates new objects (blob, tree, commit)
2. These objects are written but not yet referenced by any branch
3. Garbage collection starts and scans reachable objects
4. Since the new commit is not yet referenced, GC considers it unreachable
5. GC deletes the object
6. Commit finishes and updates HEAD to point to the deleted object

This results in repository corruption.

How Git avoids this:

1. **Atomic updates**
    - Objects are written before updating references
2. **Reference update last**
    - Branch pointers are updated only after objects are safely stored
3. **Grace period**
    - Recently created objects are not immediately deleted
4. **Locking mechanisms**
    - Prevent GC from running concurrently with critical operations